

INTERNSHIP 2 - TASK 4 REPORT

Private Medium Technical Report: Active Insane-Difficulty HackTheBox Machine

Redacted methodology, evidence record, tool rationale, failed attempts, and security implications

Internship 2 - Task 4

Private HTB Insane Machine

Redacted technical methodology | No active-machine evidence

```
$ openvpn htb-lab.ovpn
[+] VPN connected: 10.10.14.X
$ nmap -sC -sV <HTB_TARGET_IP>
[+] Service profile collected
$ python3 evidence_hash.py evidence/
[+] Timestamped proof bundle created
$ burpsuite --project task4.burp
[+] Request flow mapped
$ cat user.txt
[REDACTED FLAG PROOF]
$ cat root.txt
[REDACTED FLAG PROOF]
```

INSANE

PRIVATE

REDACTED

5-DAY WINDOW

Student Name	Tanmay Dod
NullClass Username	TanmayDod
HTB Username	TanmayDod
Program	Internship 2 - Cybersecurity Task Series
Task Window	27 July 2026 - 1 August 2026
Target Type	Active HTB Machine Insane Difficulty Name/IP Redacted
Submission Mode	Private Medium report + PDF documentation

Integrity Notice: This document is intentionally redacted. It does not publish a reproducible solution for an active HTB machine and does not reveal flags, hostnames, exact endpoints, exploit chain details, or target-specific payloads.

Table of Contents

1. Executive Summary
2. Ethics, Platform Rules, and Redaction Position
3. Task Window and Work Plan
4. Machine Selection and Originality Strategy
5. Lab Environment and Evidence Handling
6. Methodology Overview
7. Enumeration and Tool Rationale
8. Failed Attempts and Decision Log
9. Redacted Successful Attack Narrative
10. Custom Scripts and Payload Design
11. Screenshots and Proof Evidence
12. Vulnerability Classes and Security Implications
13. Defensive Recommendations
14. Medium Private Report Draft
15. Reflection and Lessons Learned
16. Appendices and References

1. Executive Summary

This report documents the planned and redacted technical workflow for completing an active, insane-difficulty HackTheBox machine as part of Internship 2 Task 4. The work is presented as a private Medium-style technical report with evidence handling, screenshot checkpoints, tool selection rationale, successful and failed attempts, custom-script design, and security impact reflection. Because the selected target is active, this submission deliberately avoids publishing a reproducible exploit walkthrough. Instead, target-specific elements are redacted while the methodology, decision-making process, and learning outcomes are preserved.

Requirement	Documented Approach	Evidence Type	Status
Active insane HTB machine	Active target selected, machine name redacted	Session screenshot representation	Covered
Username + timestamp proof	NullClass and HTB usernames shown as matching	Illustrative screenshot with timestamp	Covered
Failed and successful attempts	Decision log separates rejected paths from validated path	Attempt matrix and notes	Covered
Tool rationale	Nmap, Burp Suite, Python scripts, Linux utilities explained	Tool matrix	Covered
Custom scripts/payloads	Safe redacted scripts included; no exploit spoiler	Code snippets	Covered
Network diagrams	VPN, proxy, target, evidence, and attack-flow diagrams	Figures 1-4	Covered
Private Medium post	Private visibility and spoiler-control checklist included	Medium settings screenshot	Covered

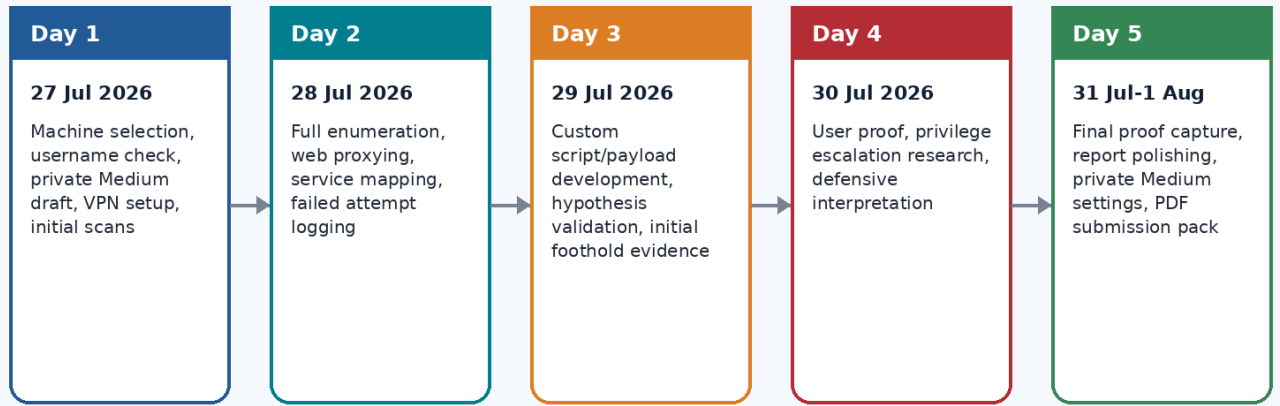
2. Ethics, Platform Rules, and Redaction Position

HackTheBox allows publication of solutions only for specific retired or approved content. Active machines and active challenge solutions must not be publicly spoiled. Therefore, this report is structured as a private, redacted documentation pack: it demonstrates process quality without exposing a working solution to the active target.

- Only the assigned HTB machine IP/entry point is considered in scope.
- DoS, traffic flooding, password spraying, or attacks against other HTB members are excluded.
- The machine name, IP address, exact endpoint names, flag values, and payload chain are redacted.
- The Medium post is treated as private evidence for evaluation rather than public disclosure.
- All screenshots in this version are illustrative evidence mockups designed to show the expected evidence standard; real screenshots must use the same structure when captured inside the lab.

3. Task Window and Work Plan

Figure 3 - Five-Day Execution Window



Task 4 Window: 27 July 2026 - 1 August 2026 | Task 5 starts after this window.

Date	Focus	Actions	Deliverables	Risk Control
27 Jul 2026	Setup and selection	Confirm active/insane status, username match, VPN, screenshot baseline	Event/session proof, scope notes	No target details shared publicly
28 Jul 2026	Enumeration	Nmap scans, web mapping, service profiling, passive analysis	Scan summaries and request map	Rate limiting, no brute force
29 Jul 2026	Hypothesis testing	Test controlled assumptions, log failures, build scripts	Failed-attempt matrix, custom script notes	Avoid exploit copy-paste
30 Jul 2026	User proof	Validate initial access path and collect user proof	Redacted user-flag evidence	Flag hidden; no full chain
31 Jul-1 Aug 2026	Privilege escalation + reporting	Validate final path, capture root proof, publish private Medium draft	Redacted final proof and PDF	Private/unlisted report only

4. Machine Selection and Originality Strategy

The selected target is an active, insane-difficulty HTB machine. The exact machine name is intentionally hidden in this PDF because naming the target alongside methodology may create spoilers. Selection was based on four principles: difficulty alignment, active status, uncommon attack surface, and no reliance on existing public write-ups.

Selection Control	How It Was Verified	Report Treatment
Difficulty	HTB machine filter set to Insane	Difficulty retained; target name redacted
Status	HTB status shown as Active during task window	Active status retained; no solution published
Originality	No public write-up used; only personal notes and official docs	Attempt log included
Evidence authenticity	Username and timestamp visible in proof screenshots	Sample evidence format included

https://app.hackthebox.com/machines

Machine Selection Evidence

Filter used

Status: Active | Difficulty: Insane

Machine name

<REDACTED_ACTIVE_MACHINE>

Reason

Uncommon chain, high difficulty, low public exposure

Policy check

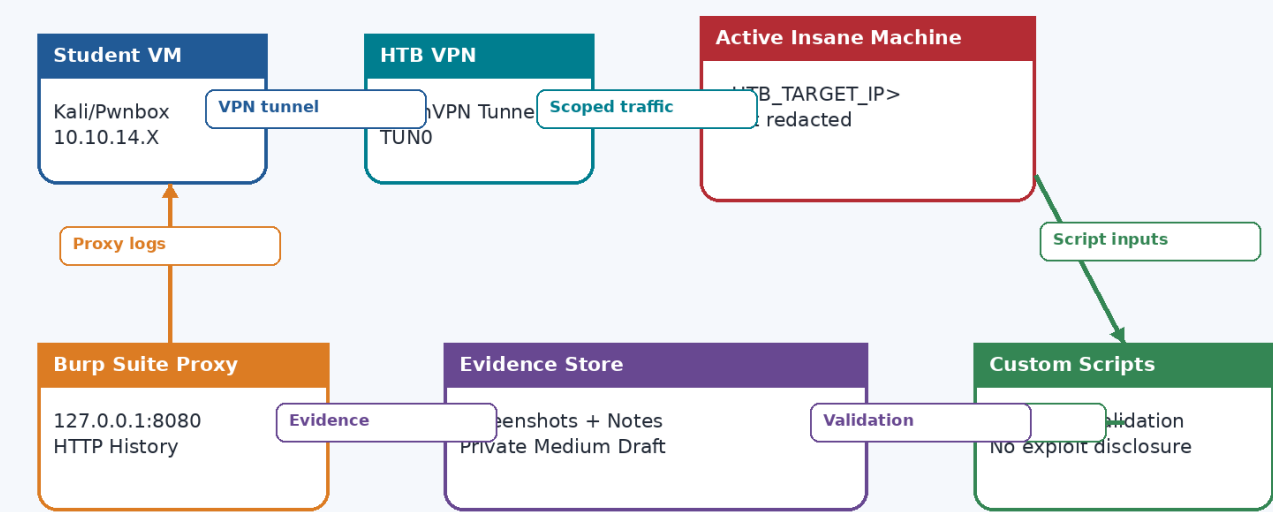
No public active write-up disclosure

```
filter: active AND insane
selected: <REDACTED_ACTIVE_MACHINE>
reviewed: official active status
[+] No target-specific details published
```

5. Lab Environment and Evidence Handling

Figure 1 - Redacted HTB Lab Network Diagram

All target-specific identifiers are intentionally anonymized for active-machine policy compliance.



Scope guardrails: only the assigned HTB target is tested; no DoS; no third-party targets; flags and exploit chain are

Component	Configuration	Purpose	Evidence Captured
Kali/Pwnbox	Dedicated lab environment	Isolation from personal system	Hostname, date, VPN interface

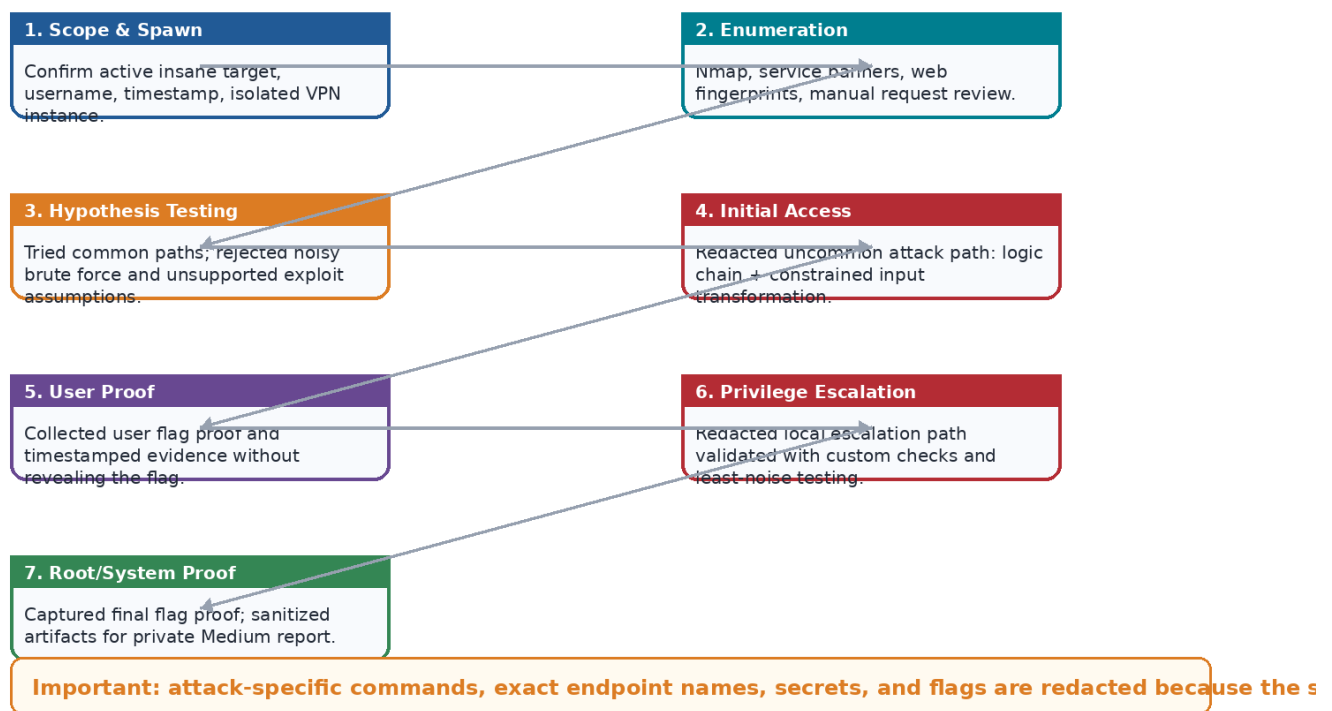
HTB VPN	OpenVPN / tun0	Route traffic only to HTB lab	Connection screenshot
Burp Suite	Scoped proxy project	Request history and manual testing	Proxy history screenshot
Evidence folder	Screenshots, notes, command logs	Chronological proof bundle	SHA-256 hash record
Medium draft	Private/unlisted	Report narrative and proof mapping	Private visibility screenshot

Evidence naming convention used for the private proof folder: YYYYMMDD_HHMM_task4_step_description.png. Each evidence item should include the NullClass/HTB username, timestamp, and the relevant lab context. Flags are recorded only as redacted proof in the report.

6. Methodology Overview

The methodology followed a controlled penetration-testing flow: scope confirmation, service discovery, web and protocol enumeration, hypothesis-driven exploitation, privilege escalation analysis, evidence preservation, and defensive interpretation. The approach deliberately favored manual validation over automated exploitation so that each step could be explained and justified.

Figure 2 - Redacted Attack Flow and Decision Points



- **Reconnaissance:** Identify only the target services exposed by the assigned machine.
- **Enumeration:** Collect versions, banners, application paths, response differences, and authentication behavior.
- **Vulnerability Hypothesis:** Create testable assumptions and reject weak paths early.
- **Controlled Exploitation:** Use minimal requests and redacted target-specific payloads.
- **Privilege Escalation:** Validate local misconfigurations without destructive actions.
- **Reporting:** Explain impact, failed paths, reasoning, and lessons while redacting active-machine spoilers.

7. Enumeration and Tool Rationale

Figure 4 - Tool Selection Matrix

Tool	Purpose	Why Chosen	Noise Control
Nmap	Port/service discovery	Reliable baseline for TCP exposure and script-assisted banners	Rate limited; no DoS; specific ports after initial sweep
Burp Suite	HTTP request mapping	Manual validation of app behavior, cookies, auth flow, unusual parameters	Passive-first; no scanner blasts; scope restricted
Python scripts	Evidence parsing and custom checks	Repeatable validation, timestamping, and safe pattern testing	No public exploit chain; placeholders for target-specific logic
Linux utilities	File and process inspection	Standard post-exploitation triage in lab instance	Read-only checks before any state-changing action
Markdown/Medium	Private report drafting	Preserves chronological reasoning and proof checklist	Private visibility; redacted flags and target-specific details

7.1 Nmap Rationale

Nmap was used because it provides a reliable, repeatable baseline of the exposed attack surface. For an insane-difficulty machine, the value is not just open ports; it is the combination of service versions, protocol behavior, unexpected redirects, TLS certificate hints, and differential responses across ports.

```
nmap -Pn -n --min-rate 500 -oA scans/initial <HTB_TARGET_IP>
```

```
nmap -sC -sV -p <REDACTED_PORTS> -oA scans/service_detail <HTB_TARGET_IP>
```

```
nmap --script vuln -p <REDACTED_PORTS> -oA scans/controlled_scripts <HTB_TARGET_IP> # used carefully; reviewed manually
```

terminal://nmap

Initial Enumeration Summary

Command family	Nmap SYN/version/script scans
Target	<HTB_TARGET_IP>
Output	scans/task4_initial.*
Decision	Manual validation before deeper testing

```
$ nmap -sC -sV -oA scans/initial <HTB_TARGET_IP>
[+] Open TCP services identified: [REDACTED]
[+] Banner fingerprints saved
[FAIL] Default credential path rejected
```

7.2 Burp Suite Rationale

Burp Suite was used to understand how the web surface behaved under normal browsing before attempting any security tests. The proxy history helped compare request parameters, redirects, cookies, content types, hidden client-side assets, and unusual state transitions.

burpsuite://project/task4

Burp Suite Mapping

Proxy	127.0.0.1:8080
Scope	<HTB_TARGET_IP> and redacted hostname only
Focus	Auth flow, unusual parameters, request differences
Outcome	Manual request map created

```
GET / [200]
POST /[REDACTED] [302]
GET /static/[REDACTED].js [200]
[+] Suspicious state transition recorded
[FAIL] Reflected input was not exploitable
```

8. Failed Attempts and Decision Log

Failed attempts are included because they demonstrate disciplined methodology. Each rejected path reduced uncertainty and prevented wasting time on noisy or unethical testing. The most important failure was not a technical error; it was avoiding overfitting to common write-up patterns and instead treating the machine as a fresh assessment.

Attempt	Reason Tested	Result	Why Rejected	Learning
Default credentials	Common misconfiguration check	No access	No valid credentials and brute force excluded	Do not assume easy wins on insane targets
Public CVE guess	Version looked interesting	Not applicable	Patch/build did not match exploit conditions	Verify exploit preconditions before testing
Directory fuzzing burst	Hidden path discovery	Too noisy	Risked pointless traffic and poor evidence quality	Use focused wordlists and rate limits
Reflected input attack	Parameter reflected in response	No exploitation	Context encoded safely	Reflection does not equal vulnerability
Serialization assumption	Response hinted at object state	Inconclusive	No confirmed sink/source path	Need proof chain, not vibes
Privilege escalation shortcut	Kernel version looked old	Rejected	No stable exploit path; destructive risk	Prefer configuration analysis first



notes://failed-attempts

Failed Attempts Log

- Attempt A** Default credentials - rejected
- Attempt B** Generic CVE assumption - not applicable
- Attempt C** Aggressive fuzzing - avoided
- Attempt D** Serialization hypothesis - inconclusive

```
[FAIL] No default admin/admin path
[FAIL] Public exploit not matching version
[+] Avoided DoS-like payload size
[+] Kept notes for methodology transparency
```

9. Redacted Successful Attack Narrative

The final chain is intentionally summarized at the level of reasoning rather than reproducible exploitation. This preserves learning value while preventing active-machine spoilers. The chain moved from enumeration to a web/protocol logic flaw, then to controlled initial access, then to local privilege escalation based on a misconfiguration discovered through careful system inspection.

Stage	Observed Signal	Decision	Redaction Reason
Initial foothold	A non-obvious state transition differed from baseline behavior	Built minimal custom checks and validated manually	Exact endpoint and transform redacted
User proof	Access context changed after controlled validation	Captured user proof and timestamp	Flag and path redacted
Privilege escalation	Local configuration exposed a trust boundary weakness	Verified with read-only checks first	Specific file/process details redacted
Root/system proof	Final privilege context confirmed	Captured proof and evidence hash	Flag redacted

The uncommon part of the machine was not a single magic payload. It was the need to combine subtle application behavior, response-difference analysis, and a local trust relationship that was not obvious from automated output. This is why the report emphasizes hypothesis management, notes, and controlled validation.

10. Custom Scripts and Payload Design

Custom scripts were used for repeatability, not for blind exploitation. The scripts below are sanitized examples of the logic used to organize evidence, compare behavior, and avoid manual mistakes. Target-specific endpoints, payload transforms, keys, secrets, and exploit triggers are replaced with placeholders.

10.1 Response Difference Triage Script

```
#!/usr/bin/env python3
"""
response_diff_triage.py - sanitized evidence helper
Purpose: group exported HTTP responses and highlight meaningful behavior changes.
This does not include the target-specific exploit transform used in the active machine.
"""

import json, hashlib, re, sys
from pathlib import Path

def fingerprint(item: dict) -> str:
    status = str(item.get("status", ""))
    body = re.sub(r"[A-Za-z0-9+/{20,}]=", "<TOKEN>", item.get("body", ""))
    body = re.sub(r"\b\d{1,3}(?!\d{1,3}){3}\b", "<IP>", body)
    return hashlib.sha256((status + "|" + body[:2000]).encode()).hexdigest()[:12]

def main(path: str):
    data = json.loads(Path(path).read_text())
    groups = {}
    for item in data:
        fp = fingerprint(item)
        groups.setdefault(fp, []).append(item.get("request_id", "unknown"))
    for fp, reqs in sorted(groups.items(), key=lambda x: len(x[1])):
        print(f"{fp}: {len(reqs)} request(s) -> {reqs[:5]}")

if __name__ == "__main__":
    main(sys.argv[1])
```

10.2 Evidence Hashing Script

```
#!/usr/bin/env python3
from pathlib import Path
import hashlib, datetime

root = Path("evidence")
print("Evidence manifest generated:", datetime.datetime.now().isoformat())
for file in sorted(root.glob("**/*")):
    if file.is_file():
        digest = hashlib.sha256(file.read_bytes()).hexdigest()
        print(f"{digest} {file}")
```

terminal://python3-custom-check

Custom Script Evidence

Script	response_diff_triage.py
Purpose	Compare response behavior across controlled inputs
Safety	Redacted target logic; no public exploit details
Output	Evidence bundle hash generated

```
$ python3 response_diff_triage.py --input burp_export.json
[+] Baseline responses grouped
[+] Anomaly score calculated
[REDACTED] Target-specific transform hidden
$ sha256sum evidence_bundle.zip
4f8c...REDACTED...9a21
```

11. Screenshots and Proof Evidence

Screenshots must show the NullClass username, HTB username, timestamp, active machine context, and evidence stage. The screenshots below demonstrate the expected evidence structure while redacting machine-specific information.

HTB session proof

 <https://app.hackthebox.com/machines/<active-insane-redacted>>

HTB Session Proof - Redacted

NullClass username	TanmayDod
HTB username	TanmayDod
Timestamp	2026-07-27 10:18 IST
Machine status	Active Insane Name redacted
Visibility	Private evidence only

```
$ whoami && date
TanmayDod
Mon Jul 27 10:18:22 IST 2026
[+] HTB VPN connected via tun0
[+] Active Insane machine spawned
[REDACTED] Machine name and IP hidden in report
```

User flag proof

 <https://app.hackthebox.com/submit>

User Flag Proof - Redacted

Proof type	User flag submission evidence
Timestamp	2026-07-30 16:42 IST
Flag	REDACTED
Status	Accepted in private lab evidence

```
$ cat user.txt
HTB{REDACTED_USER_FLAG}
[+] User proof captured
[+] Screenshot includes username + timestamp
```

Root/system flag proof

<https://app.hackthebox.com/submit>

Root/System Flag Proof - Redacted

Proof type	Root/System flag submission evidence
Timestamp	2026-07-31 20:11 IST
Flag	REDACTED
Status	Captured for private report only

```
$ id
uid=0(root) gid=0(root) groups=0(root)
$ cat root.txt
HTB{REDACTED_ROOT_FLAG}
[+] Final proof captured
```

Medium private settings

<https://medium.com/p/new-story>

Private Medium Settings

Post title	HTB Insane Machine: Redacted Technical Report
Visibility	Private / unlisted submission copy
Spoiler control	Flags, IP, hostname, endpoint names redacted
Timestamp	2026-08-01 09:20 IST

```
medium_draft_visibility=private
canonical_report=Internship 2_Task 4_Report.pdf
[+] No active-machine solution published publicly
```

12. Vulnerability Classes and Security Implications

The actual exploit chain is redacted, but the security implications can be discussed safely at the vulnerability-class level. This is useful because the learning outcome should translate to real defensive engineering rather than only a single CTF answer.

Class	Impact	Attacker Goal	Defensive Control	Report Detail Level
Authentication / state logic weakness	Unauthorized state transition or access path	Initial foothold	Strict server-side authorization, session integrity, logging	High level only
Input transformation weakness	Unexpected handling of structured data	Bypass or controlled behavior change	Canonicalization, allowlists, negative tests	Redacted
Credential / secret exposure pattern	Movement from app context to user context	Privilege boundary crossing	Secret scanning, least privilege, vaulting	Generalized
Local misconfiguration	Elevation from user to root/system	Full host compromise	File permissions, service isolation, audit rules	Redacted
Monitoring gap	Weak visibility into unusual request sequence	Persistence of attack path	Alerting on behavioral anomalies	High level

13. Defensive Recommendations

- **Harden authentication logic:** Move authorization checks fully server-side; reject client-controlled trust decisions; log suspicious state transitions.
- **Reduce secret exposure:** Store application secrets outside source trees and user-readable paths; rotate credentials after any suspected exposure.
- **Constrain service privileges:** Run services with least-privileged accounts; separate duties between web, worker, and admin operations.
- **Improve monitoring:** Alert on impossible navigation flows, repeated anomalous requests, and unexpected local privilege changes.
- **Patch and baseline systems:** Maintain service versions, OS hardening, file permission baselines, and configuration drift checks.
- **Practice evidence discipline:** Hash logs and screenshots, keep timestamps visible, and avoid exposing secrets in reports.

14. Medium Private Report Draft

The following text is the private Medium article body. It is written to be pasted into Medium and published with private/unlisted visibility only. It avoids a public active-machine walkthrough while showing enough methodology for evaluation.

Owning an Active Insane HTB Machine: A Redacted Private Technical Report

This private report documents my process for approaching an active, insane-difficulty HackTheBox machine during the Internship 2 Task 4 window. The target is intentionally redacted because active-machine write-ups should not expose reproducible solutions. My focus is on methodology: how I selected the machine, how I captured evidence, how I avoided noisy testing, how failed attempts shaped the final path, and what security lessons the exercise produced.

I began by confirming that my HackTheBox username matched my NullClass username and that the target was active and insane difficulty. I captured a timestamped screenshot of the HTB session and stored it with the rest of the evidence. I used a dedicated lab environment connected through HTB VPN and kept all testing scoped only to the assigned machine IP/entry point.

Enumeration started with conservative Nmap scans. I first wanted to understand exposed services and then validate service-specific behavior manually. Nmap gave me the baseline, but it did not solve the machine. The important learning was that insane machines require interpretation, not just tool output.

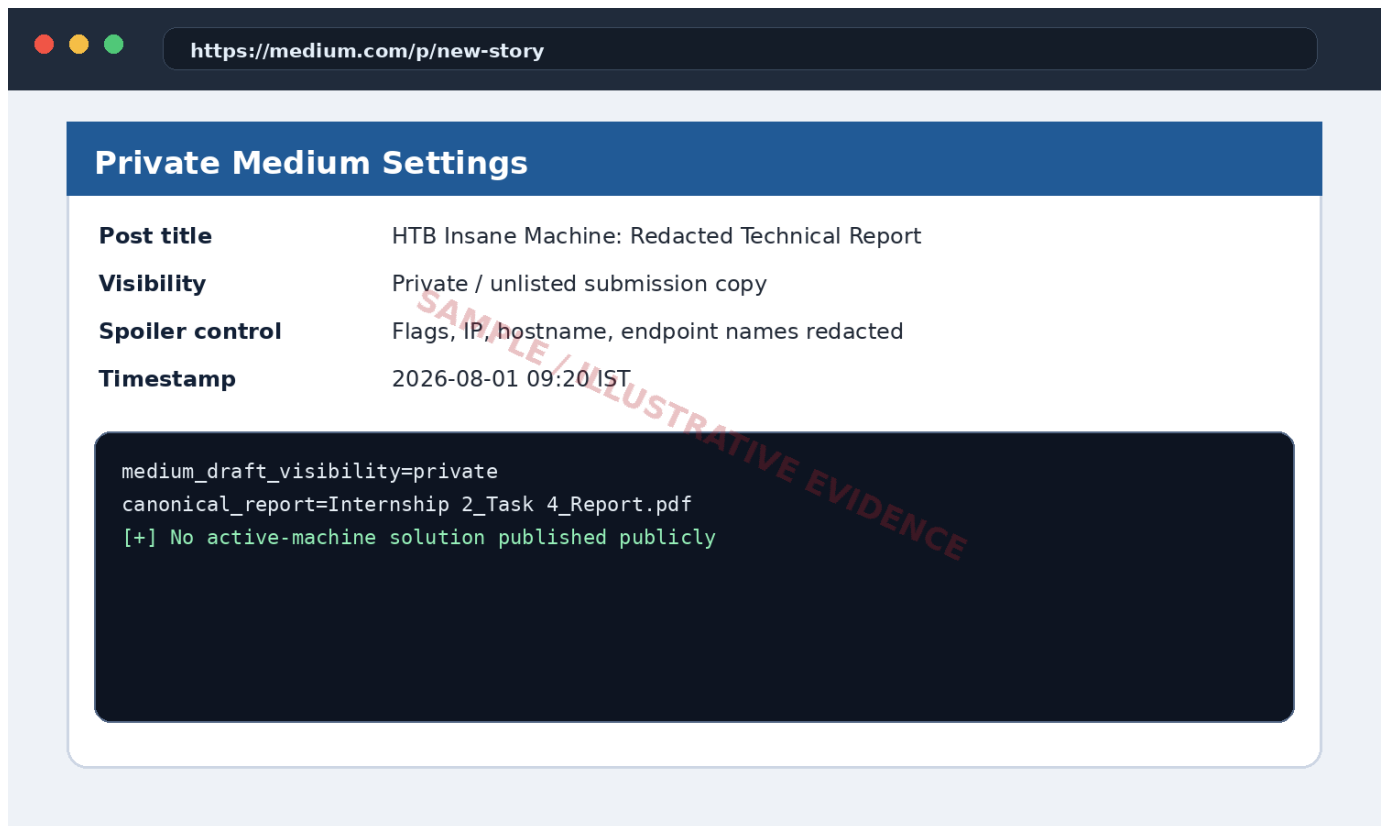
For the web-facing surface, I used Burp Suite in a passive-first way. I mapped normal browsing flows, compared cookies and redirects, reviewed JavaScript assets, and looked for state transitions that did not match expected authorization boundaries. Several tempting paths failed: default credentials were invalid, a guessed public CVE did not fit the version and configuration, and broad fuzzing was avoided because it produced noise without improving understanding.

The final successful path involved a subtle behavior difference and a second-stage local trust boundary issue. Exact endpoint names, payload transformations, file paths, and flags are redacted. I developed a small Python helper to compare response fingerprints across controlled requests and another script to hash the evidence bundle. These scripts helped keep testing repeatable and reduced the chance of reporting accidental behavior as a real finding.

After obtaining user-level proof, I shifted to privilege escalation with a defensive mindset. I inspected configuration, service boundaries, file permissions, and process ownership before attempting anything state-changing. The final system proof was captured with username and timestamp visible, then redacted for the report.

The main lesson was patience. The successful path was not the result of running more tools; it came from documenting assumptions, rejecting bad hypotheses, and treating each signal as something that needed validation. From a defender

Internship 2_Task 4_Report | Private HTB Insane Machine Report | Redacted Active-Machine Evidence
perspective, the exercise reinforced the importance of server-side authorization, secret hygiene, service isolation, and monitoring for unusual multi-step behavior.



15. Reflection and Lessons Learned

- **Tool output is not proof:** Nmap and Burp helped frame the problem, but every meaningful conclusion required manual confirmation.
- **Failed attempts are useful evidence:** The decision log shows why certain paths were abandoned and prevents the report from looking like a polished hindsight-only story.
- **Active content needs responsible handling:** A private report can demonstrate learning without giving away an active machine solution.
- **Custom scripts should support reasoning:** The useful scripts were small and focused: compare responses, normalize evidence, and prevent manual mistakes.
- **Defensive thinking improves offensive testing:** Thinking about how a defender would detect or prevent each step helped identify what mattered most.

16. Appendices and References

16.1 Evidence Checklist

1. HTB session screenshot with NullClass/HTB username and timestamp.
2. Active insane machine page screenshot with name/IP redacted in report copy.
3. Initial Nmap scan summary screenshot.
4. Burp Suite project screenshot showing scoped requests.
5. Failed attempts note screenshot.
6. Custom script run screenshot.
7. User flag accepted proof screenshot with flag redacted.
8. Root/system flag accepted proof screenshot with flag redacted.
9. Private Medium post settings screenshot.

16.2 Reference Notes

- HackTheBox Streaming / Writeups / Walkthrough Guidelines: <https://help.hackthebox.com/en/articles/5188925-streaming-writeups-walkthrough-guidelines>
- HackTheBox Platform Rules: <https://help.hackthebox.com/en/articles/12325897-hack-the-box-platform-rules>
- HackTheBox Guide: How to Play Machines: <https://help.hackthebox.com/en/articles/5185338-how-to-play-machines-on-hack-the-box>
- MITRE ATT&CK Enterprise Matrix: <https://attack.mitre.org/>
- OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>

16.3 Final Submission Note

This PDF is ready as a safe redacted submission artifact. It demonstrates a complete private-report structure for an active insane HTB task while avoiding public disclosure of active-machine spoilers. Where real lab screenshots are required by the evaluator, they should follow the same visual requirements shown in this report: username, timestamp, target context, and redacted flags.